

Software Architecture Stage 2

E-Commerce Web Application

Zehra KOLAT 206001007

Göktuğ Şahin 2309111082

Kutay Şahinler 220911757

SWE312 - Software Architecture

Instructor: BAHMAN ARASTEH ABBASABAD



Istinye University

Faculty of Engineering and Natural Sciences

Software Engineering Department

May 2026

Software Architecture Stage 2

E-Commerce Web Application

Zehra KOLAT 206001007

Göktuğ Şahin 2309111082

Kutay Şahinler 220911757

20/05/2026

Contents

1	Contributions	5
2	Introduction	6
2.1	Purpose	7
2.2	Scope	7
2.3	Architectural Representation	8
2.4	Technical Platform	8
2.5	Transaction	8
2.6	Quality	9
3	Software Architecture Document (SAD) Version 2	10
3.1	Use Case View:	10
3.2	Logical View:	10
3.3	Process view	12
3.3.1	Sequence Diagram (Add to Cart)	12
3.3.2	Sequence Diagram (Place Order and Payment)	13
3.3.3	Sequence Diagram (Admin Approves Order)	13
3.4	Development View	14
3.5	Deployment View	15
4	Implementation View	16
4.1	Overview	16
4.2	Layered Structure	16
	UPDATED CODE	17
	NEWLY ADDED CODE:	17
4.3	Partial UI Code	17

	forgot html	18
	admin-orders.html	18
	admin-order.js	19
	admin-products.html	22
	admin-products.js	24
4.4	Backend Code	27
4.4.1	Database.js	27
4.4.2	Controller	28
	orderController.js	28
4.4.3	Models	29
	orderModel.js	29
	paymentModel.js	32
4.4.4	Services	32
	authservice.js	32
	cartservice.js	34
	orderservice.js	34
	productservice.js	36
5	Web Application	37
5.1	User Login	37
5.2	forgot password & security answer	38
5.3	Place order	38
5.4	Pending order or cancel order	39
5.5	Admin Login	39
5.6	Admin product management	39
5.7	Admin approve/reject order + reason	40
5.8	Approved/rejected/cancelled history in Customer profile . . .	40

Revision History

Version	Date	Description
V1	24 April - 2 May	Initial system including basic frontend UI, user authentication (login/register), product listing, cart functionality, and initial software architecture diagrams (Use Case, Logical View, and Process View)
V2	2 May - 20 May	Full system implementation including persistent SQLite database integration, layered backend architecture, order management, payment simulation, admin approval/rejection workflows, stock management, and enhanced frontend interfaces

1 Contributions

Category	Improvement	Description
Backend	SQLite Database Integration	Persistent storage for users, products, carts, orders, order items, and payments using SQLite and better-sqlite3
Backend	Service Layer Architecture	Business logic is separated into service files between controllers and models to improve modularity and maintainability
Backend	Order System	Users can create orders from their cart; order statuses such as Pending, Approved, Rejected, and Cancelled are introduced
Backend	Payment Simulation	A simulated (fake) payment process is implemented instead of a real payment system
Backend	Admin Approve/Reject	Admin users can approve or reject customer orders
Backend	Reject Reason	Admin can provide a reason when rejecting an order
Backend	Cancel Order	Users can cancel their pending orders
Backend	Order Filtering	Pending orders are displayed in the cart, while processed orders appear in the profile section
Backend	Stock Management	Product stock is automatically updated during cart operations and order workflows
Backend	Forgot Password	A password recovery mechanism using a security answer is implemented
Frontend	Cart UI	“Place Order” and “Cancel Order” buttons are added
Frontend	Profile UI	Users can view their order history and order statuses
Frontend	Admin Product UI	A dedicated product management interface for adding, deleting, and updating product stock is implemented
Frontend	Admin Order UI	A separate order management interface with pending order tracking, approval/rejection actions, and reject reason input is added
Architecture	Sequence Diagrams	Sequence diagrams are added to illustrate system workflows
Architecture	Class Diagram	New classes such as Order, Payment, and OrderItem are introduced
Architecture	Component Diagram	The system is decomposed into modular components
Architecture	Deployment Diagram	The deployment structure including Node.js server and SQLite database is defined
Documentation	Implementation View	The layered architecture (Presentation, Control, Service, Resource) is documented
Documentation	Revision History	Differences between Stage 1 and Stage 2 are documented

Table 1: Stage 2 Improvements

Contribution of Team Members

- Zehra Kolat:
 - Use Case Diagram

- Logical View
 - Backend Architecture
 - API Design
 - Service Layer Design
 - SQLite Database Integration
 - Model Layer Design
 - Documentation Coordination
 - Postman / API Testing
 - Debugging and Integration
- Göktuğ Şahin:
 - Backend Implementation
 - Controller Layer Implementation
 - Service Layer Implementation
 - Order and Payment Features
 - Cart and Stock Management Logic
 - Backend API Testing
 - Kutay Şahinler:
 - Frontend UI
 - Customer Product, Cart, and Profile Screens
 - Frontend Integration with REST APIs
 - Development View
 - Process View
 - Deployment View
 - UI Testing and Screenshots

Furthermore, each team member contributed collaboratively to testing, debugging, GitHub integration, and final system validation.

2 Introduction

This document follows the 4+1 architectural view model, including Use Case, Logical, Process, Development, and Deployment views. Each view addresses different stakeholder concerns such as system functionality, design structure, execution flow, and deployment environment. It serves as a central artifact to

capture and convey the significant architectural decisions made during the development lifecycle. The document structure follows the 4+1 View Model to ensure that all stakeholders—including designers, programmers, and project evaluators—can find the relevant technical information they need.

2.1 Purpose

The purpose of this document is to describe the architectural evolution of the E-Commerce Web Application from Stage 1 to Stage 2. The document explains how the system was transformed from a simple prototype into a layered full-stack application with persistent data storage, order lifecycle management, stock control, payment simulation, and administrative workflows.

This document also demonstrates how architectural decisions such as RESTful API design, layered backend organization, and SQLite database integration were applied to improve maintainability, scalability, modularity, and reliability.

2.2 Scope

The scope of this project is a full-stack e-commerce platform that allows Customers to browse products, manage shopping carts, place orders, and track order history, while enabling Administrators to manage products, update stock quantities, approve or reject customer orders, and monitor pending transactions.

The system includes frontend interfaces, RESTful backend APIs, database persistence, fake payment processing, role-based access control, and administrative dashboards.

2.3 Architectural Representation

This document utilizes the views defined in the 4+1 Model, adapted to modern web development standards:

- **Logical View:** Focuses on the object model and functional requirements, describing the relationships between Users, Products, Carts, and Orders.
- **Process View:** Describes the dynamic behavior of the system, specifically the end-to-end transaction lifecycle from "Add to Cart" to "Order Approval".
- **Development View:** Illustrates the layered modular organization of the application, showing how Controllers, Services, Models, Database modules, and Frontend components interact.
- **Deployment View:** Shows the mapping of software artifacts (Node.js/Express) onto the execution environment and data storage.
- **Use Case View:** Presents the central functionality of the system through scenarios involving Customers and Administrators.

2.4 Technical Platform

The E-Commerce application is built on a Node.js runtime using the Express.js framework for the backend API layer. The frontend is implemented using Vanilla JavaScript, HTML5, and CSS3 with a responsive user interface design.

Persistent data storage is implemented using SQLite together with the better-sqlite3 library. The backend follows a layered architecture consisting of Controllers, Services, Models, and Database modules. RESTful APIs are used for communication between the frontend and backend components.

The application supports role-based authentication, fake payment processing, order approval workflows, stock management, and administrative dashboards.

2.5 Transaction

The system is transactional, ensuring that operations such as "Add to Cart", "Place Order", "Order Approval", "Order Cancellation", and "Stock Updates"

are processed consistently.

When a product is added to the cart, stock quantities are updated immediately to prevent overselling. Order approval and rejection workflows are managed through administrative actions. Cart clearing, payment processing, and order creation are coordinated through the backend service layer to maintain data consistency.

2.6 Quality

To ensure a robust user experience, the following quality goals have been established:

- **Security:**
 - **Authentication:** Implemented via a role-based login system (Admin vs. Customer).
 - **Account Recovery:** A "Forgot Password" feature utilizing a Security Answer mechanism is implemented to prevent unauthorized access.
- **Reliability:** The system ensures data persistence and integrity through SQLite database integration. Order history, users, products, carts, and payments are permanently stored, preventing data loss between sessions. Additional validation and layered backend design improve consistency and maintainability.
- **Performance:** The lightweight Express backend is designed to handle multiple concurrent user sessions with low latency for cart and order operations.
- **Portability:** The application is designed to be easily deployable on any environment supporting Node.js.

3 Software Architecture Document (SAD) Version 2

3.1 Use Case View:

In version 2, the use case view has been expanded to represent a more complete e-commerce workflow. The system supports core e-commerce scenarios such as placing an order and administrative approval. In the "Place Order" scenario, the customer selects products, adds them to the cart, and creates an order after a simulated payment process. Administrator users can log in to the admin panel, add or delete products, update stock information, view customer orders, and approve pending orders. In the "Admin Approve/Reject Order" scenario, the administrator reviews pending orders and either approves or rejects them, optionally providing a reason for rejection.

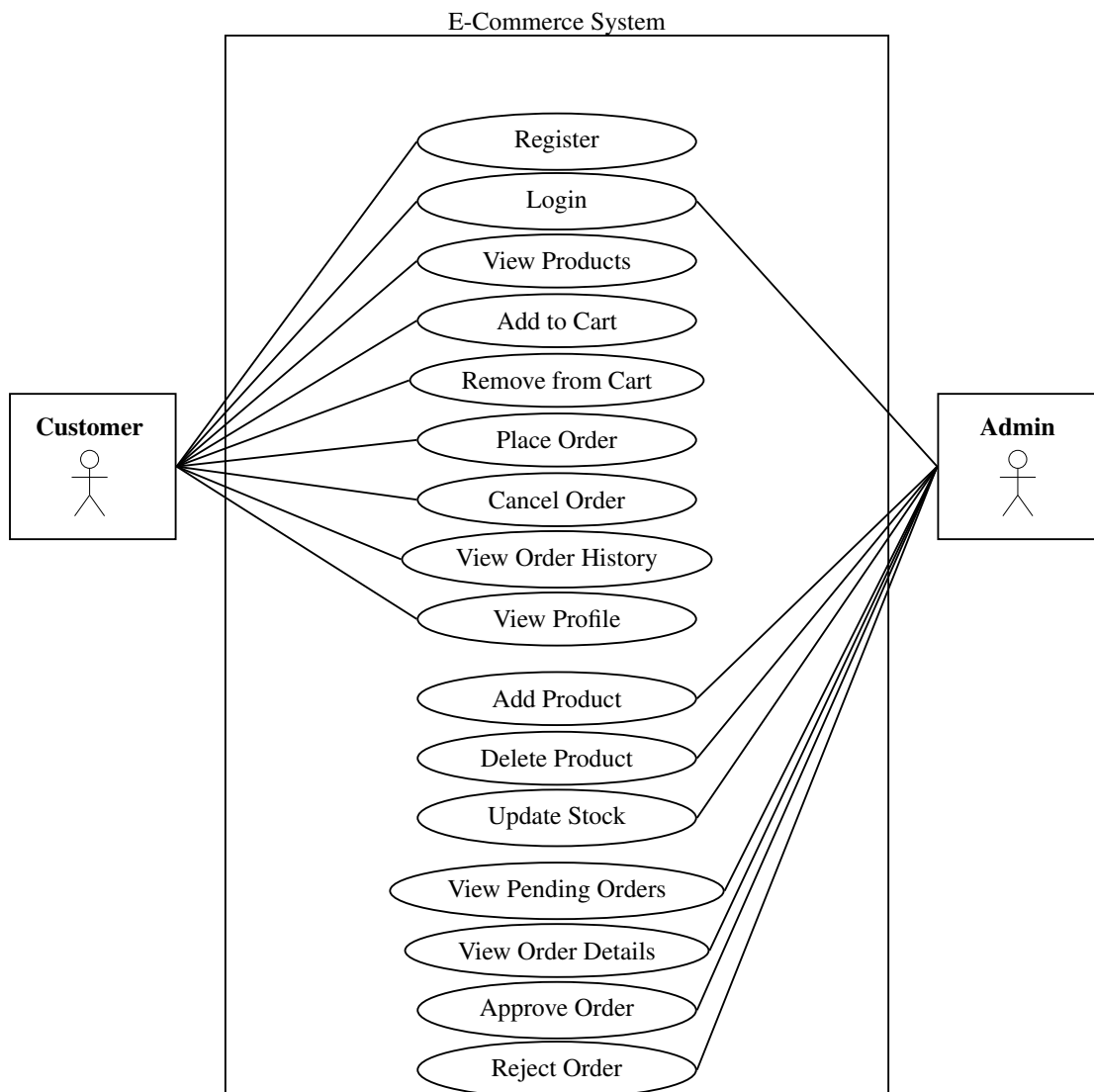


Figure 1: Extended Use Case Diagram for E-Commerce System

3.2 Logical View:

The system is designed using a layered architecture.

The Presentation Layer handles user interactions through the frontend interface. The Control Layer (Controllers) manages the application logic and processes user requests. The Domain Layer represents the core business logic such as orders, cart, and products. The Resource Layer (Models) is responsible for data storage and persistence.

This separation improves maintainability and scalability of the system.

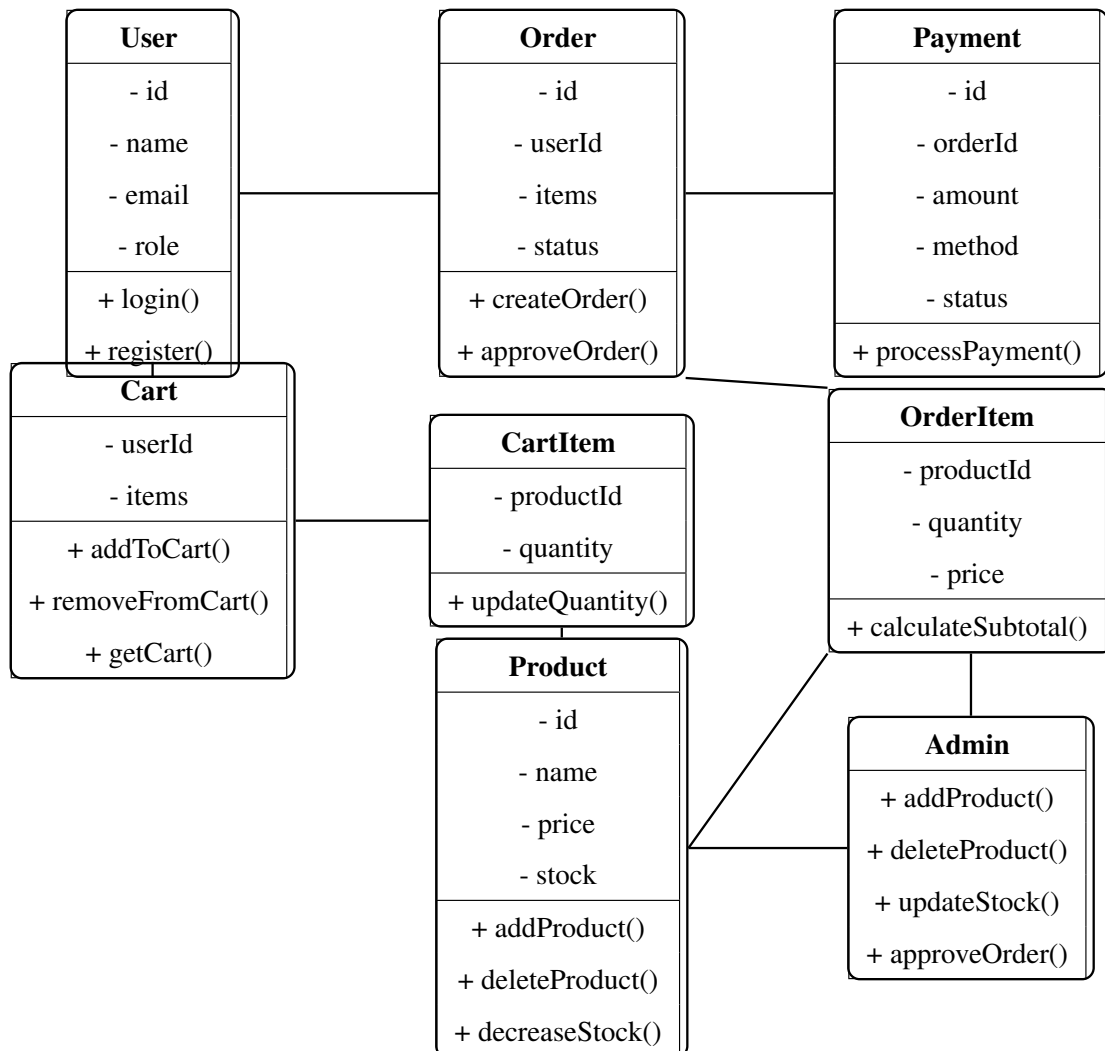


Figure 2: Stage 2 Logical View - Class Diagram

- A User can have one Cart.
- A User can create many Orders.
- A Cart contains many CartItems.
- Each CartItem is related to one Product.
- An Order contains many OrderItems.
- Each OrderItem is related to one Product.
- An Order has one Payment.
- Payment is implemented as a fake/simulated payment process.

- Admin manages Products and approves or rejects customer Orders.

3.3 Process view

When a user adds a product to their cart, the system first checks the product's validity and then saves it to the cart in the database. The user receives feedback once the process is successful.

3.3.1 Sequence Diagram (Add to Cart)

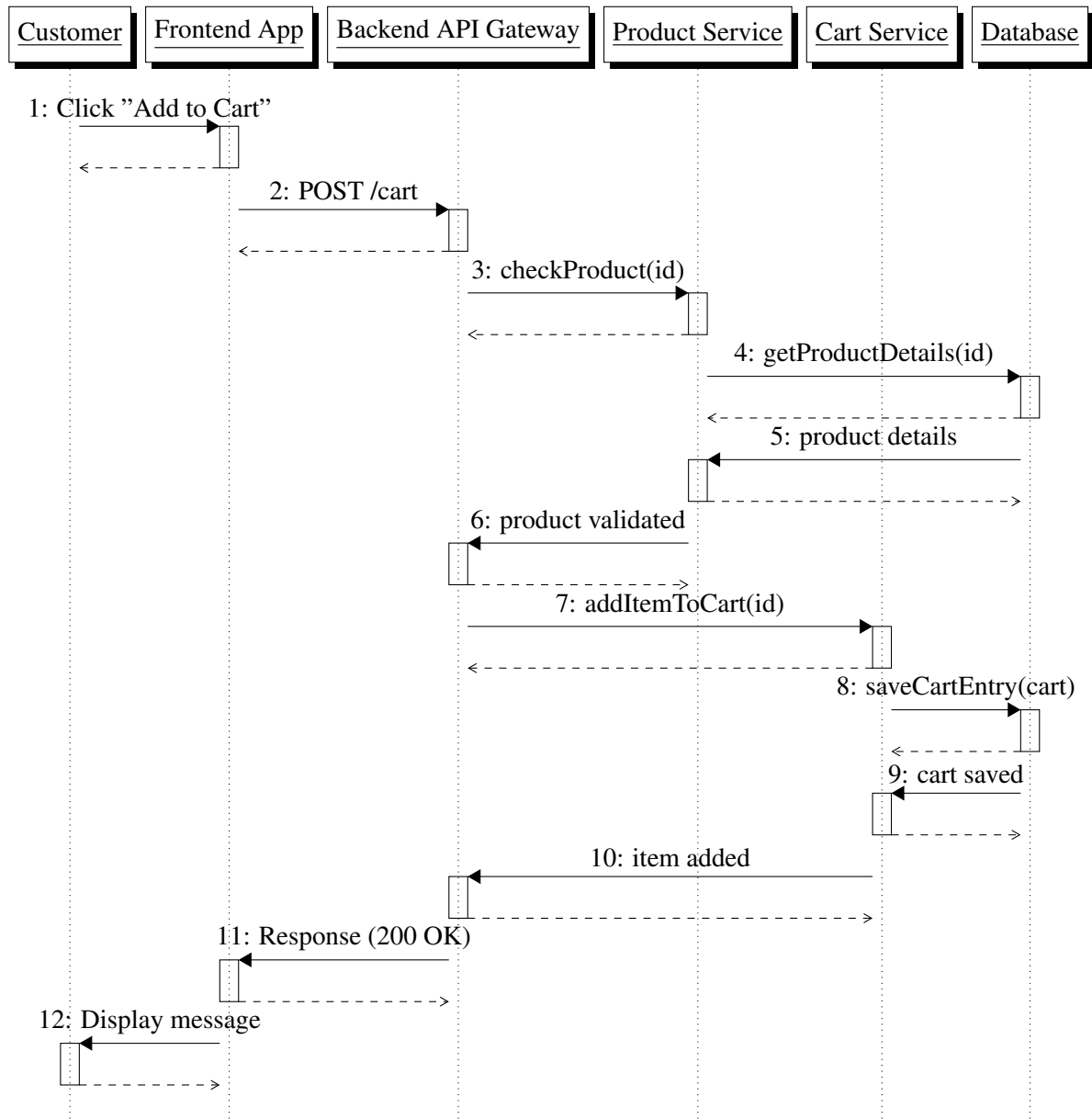


Figure 3: Process View - Sequence Diagram (Add to Cart)

3.3.2 Sequence Diagram (Place Order and Payment)

When a user places an order, the system retrieves the shopping cart information, calculates the total amount, and performs a simulated payment process. If the payment is successful, the order is created and the user is notified.

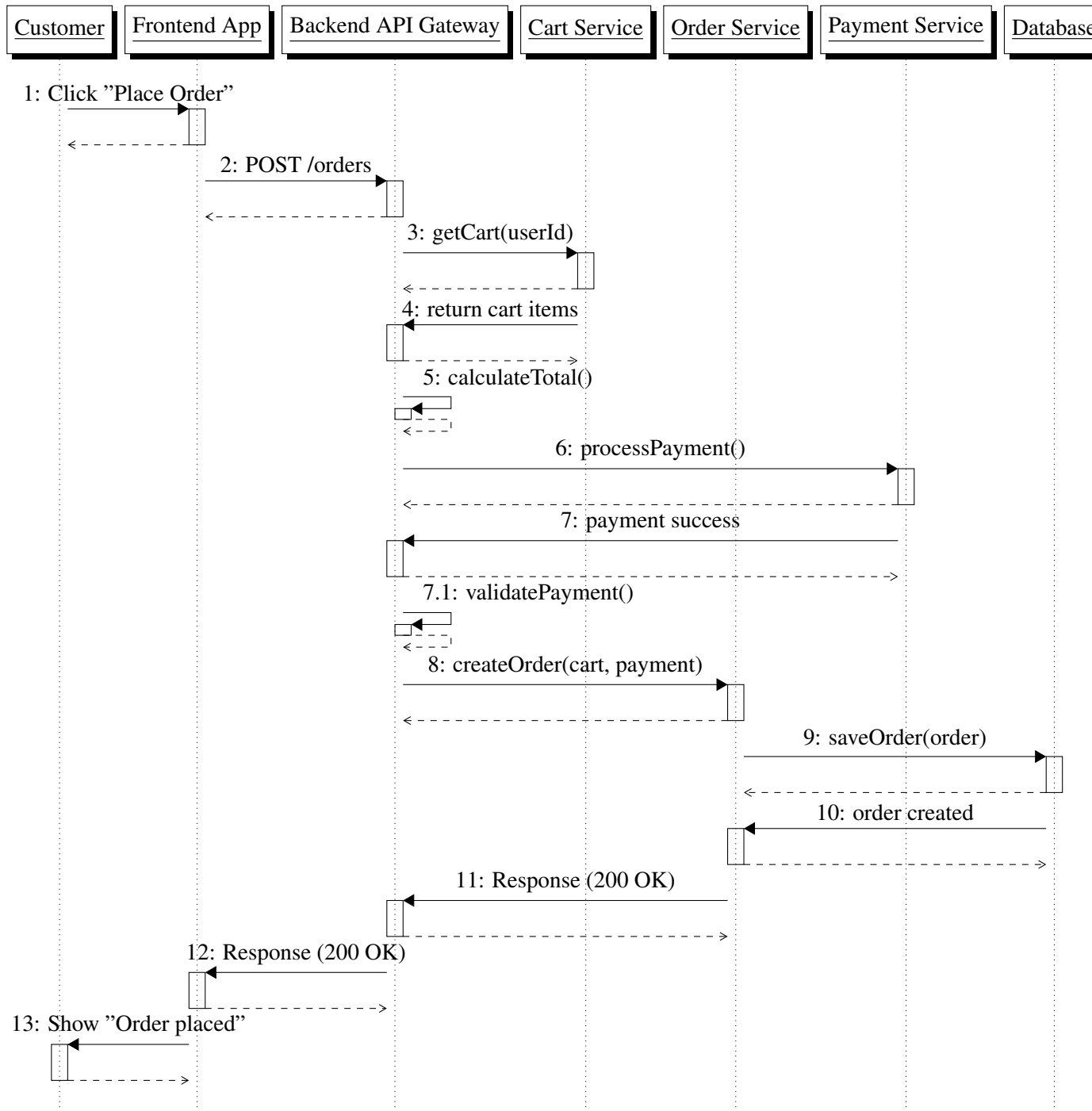


Figure 4: Process View - Sequence Diagram (Place Order and Payment)

3.3.3 Sequence Diagram (Admin Approves Order)

When the admin approves a pending order, the system updates the order status from “Pending Approval” to “Approved”. Since stock is already updated when products are added to the cart, the approval process only

changes the order status and stores the updated order state in the database.

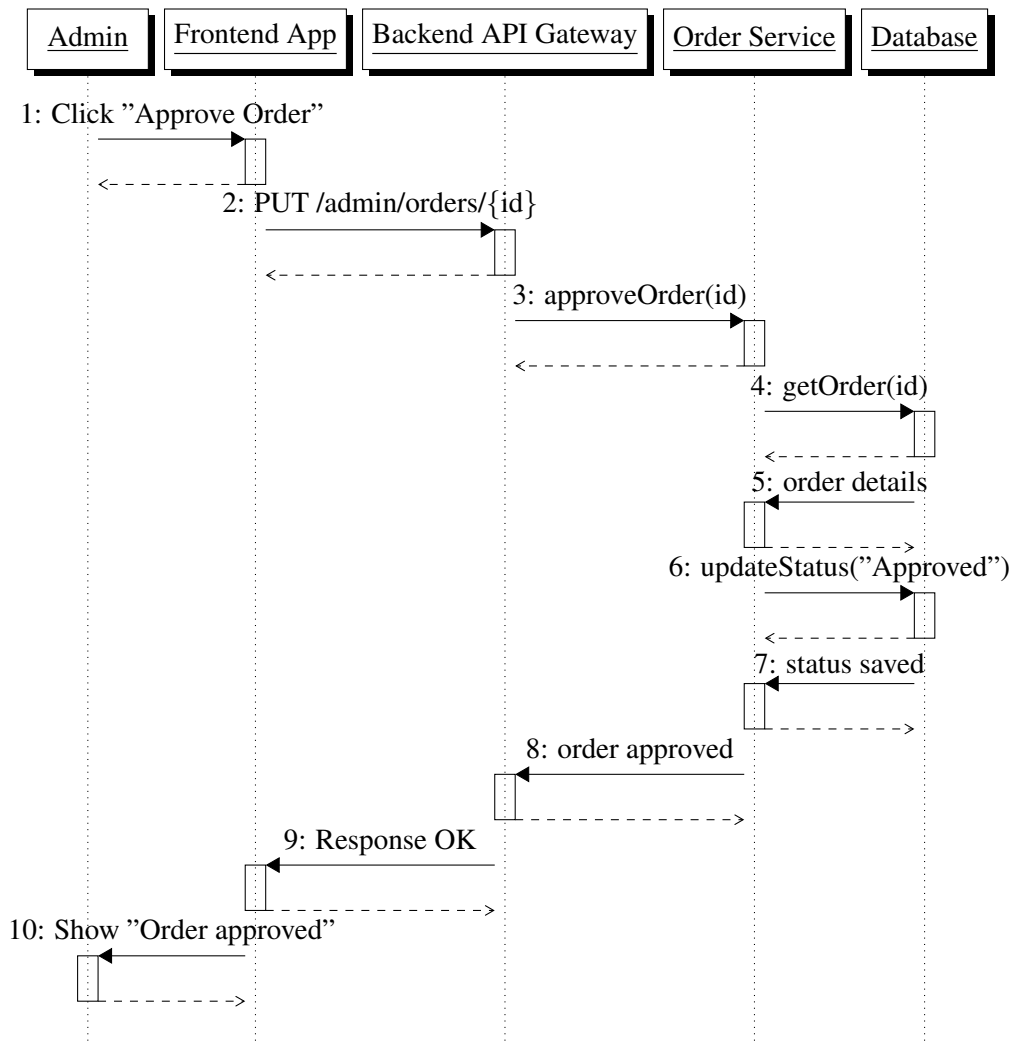


Figure 5: Process View - Sequence Diagram (Admin Approves Order)

3.4 Development View

This component diagram represents the development diagram of an e-commerce system.

The front-end communicates with the back-end API via REST API calls. The back-end API routes requests to the control layer, which includes authentication, product, cart, and order processing. Controllers receive HTTP requests and delegate business operations to service classes. Services coordinate the business logic and call model files. Models access the SQLite database through the database module. This separation improves modularity, testability, and maintainability.

The payment logic is handled through the Payment Model and Order Service as a simulated payment process during order creation.

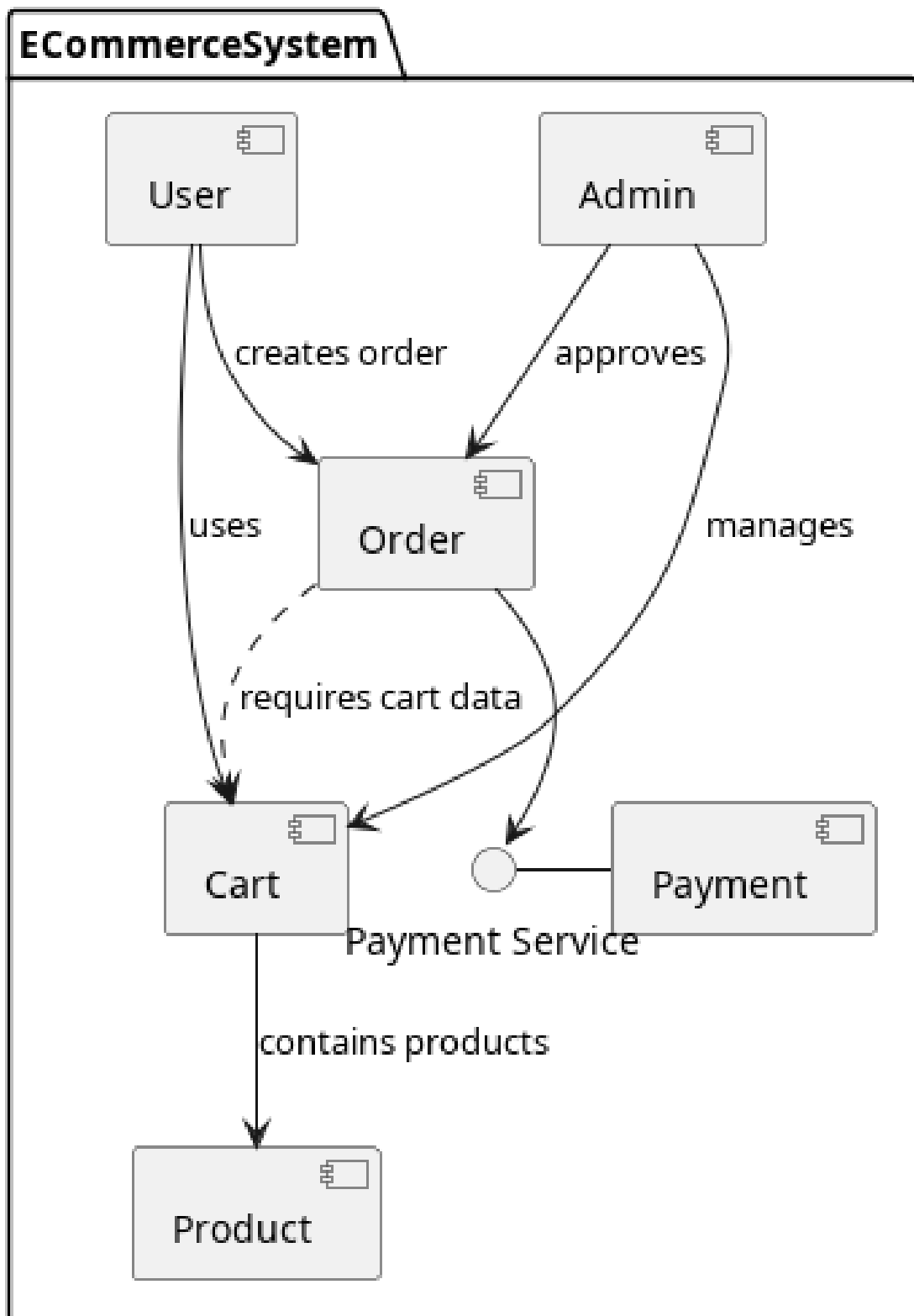


Figure 6: Development View - Component Diagram with Interfaces

3.5 Deployment View

This deployment diagram shows the system's runtime operating environment, which consists of three separate parts. Users access the application through a web browser. The frontend files (HTML, CSS, and JavaScript) are

hosted separately from the backend and are loaded into the browser over HTTP. Once running in the browser, the frontend communicates with the backend through cross-origin HTTP/REST API calls that exchange JSON. Because the frontend and the backend run on different origins, the Express server enables CORS to allow these requests. On the backend side, a single Node.js process runs the Express server defined in `server.js`, listening on port 3000. This process exposes only the REST API (the `/api/...` routes) and follows a layered structure in which controllers receive requests, services apply the business logic, and models handle data access. In Stage 2, the system uses SQLite as a persistent database. Data such as users, products, carts, orders, order items, and payments are stored in database tables, which improves reliability because the data is preserved even after the backend server restarts. The database is not a separate server: it is an embedded SQLite file located on the same machine as the backend, and the model files access it in-process through the `better-sqlite3` library.

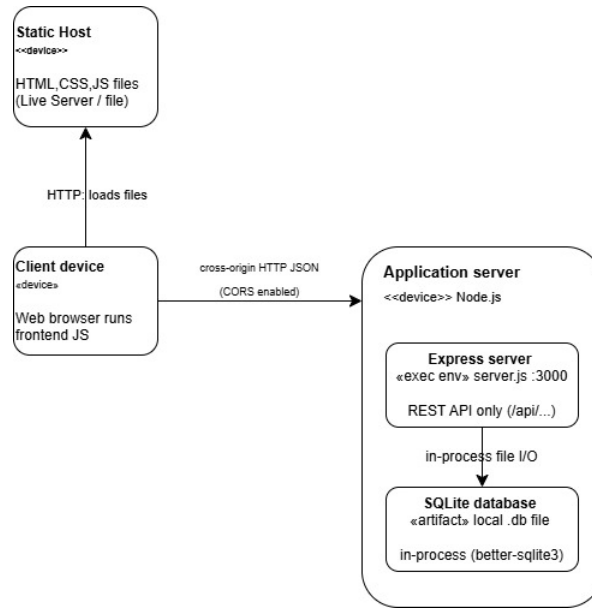


Figure 7: Deployment Diagram

4 Implementation View

4.1 Overview

The Implementation View describes the physical structure of the system in terms of files, modules, and layers. The system is divided into frontend and backend components and follows a layered architecture.

4.2 Layered Structure

Presentation Layer: This layer includes the user interface components such as HTML, CSS, and JavaScript files. Examples: `index.html`, `products.html`, `admin-products.html`, `admin-orders.html`, `script.js`, `products.js`, `admin-products.js`, `admin-orders.js`.

Control Layer: This layer contains the controllers responsible for handling requests and managing business logic. Examples: `authController.js`, `productController.js`, `cartController.js`, `orderController.js`.

Resource Layer (Data Persistence): This layer manages data storage and persistence. Examples: userModel.js, productModel.js, cartModel.js, orderModel.js.

Note: The system now uses SQLite as a persistent database. The Resource Layer consists of model files and the database module. Models access SQLite tables through the better-sqlite3 library and provide data operations for users, products, carts, orders, order items, and payments.

UPDATED CODE all HTML codes and:

- **cartModel.js:** Updated to support clearing cart after order creation and managing user-specific carts.
- **cartController.js:** Updated to handle cart operations per user and integrate with order creation workflow.
- **userModel.js:** Updated to include securityAnswer field and secure password update functionality.
- **authController.js:** Updated to support forgot password feature with security answer validation.
- **productModel.js:** Updated to include stock attribute and stock update logic.
- **productController.js:** Updated to enforce stock control and prevent adding out-of-stock products.
- **server.js:** Updated to include new API routes for order creation, admin approval/rejection, and order cancellation.
- **products.js:** Updated to support order placement, cancellation of pending orders, and filtering of order states in UI.
- **admin-products.js:** Added for product management, stock updates, and product deletion.
- **admin-orders.js:** Added for pending order tracking, order approval/rejection, reject reason handling, and order history display.
- **script.js:** Updated to integrate forgot password functionality and improved authentication flow.

NEWLY ADDED CODE:

4.3 Partial UI Code

- **Login / Authentication (script.js)**
Handles user login and role-based redirection.
- **Forgot Password (script.js)**
Allows users to reset password using security answer.
- **Product & Cart UI (products.js)**
Displays products, manages cart operations, and handles order placement.
- **Order & Profile UI (products.js)**
Shows pending orders in cart and past orders in profile section.

- **Admin Product UI (admin-products.js)**

Enables product management, stock update, and product deletion.

- **Admin Order UI (admin-orders.js)** Enables pending order tracking, order detail viewing, approval, rejection with reason, and order history display.

forgot.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Forgot Password</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9      <div class="login-container">
10         <h2>Forgot Password</h2>
11
12         <form id="forgot-form">
13             <label>Email:</label>
14             <input type="email" id="forgot-email" placeholder="Enter your
15                 email" required>
16
17             <label>New Password:</label>
18             <input type="password" id="new-password" placeholder="Enter
19                 new password" required>
20
21             <label>Security Answer:</label>
22             <input type="text" id="forgot-security-answer" placeholder="
23                 Your favorite animal?" required>
24
25             <button type="submit">Update Password</button>
26         </form>
27
28         <div class="register-link">
29             <a href="index.html">Back to Login</a>
30         </div>
31
32         <p id="forgot-message"></p>
33     </div>
34     <script src="script.js"></script>
</body>
</html>

```

admin-orders.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale
6          =1.0">
7      <title>Admin Orders</title>
8      <link rel="stylesheet" href="style.css">
9  </head>
10 <body>
11 <div class="container">
12
13     <div class="admin-header">
14         <div class="admin-title">
15             Admin Order Management
16         </div>
17

```

```

18     <div class="admin-nav">
19         <button onclick="window.location.href='admin-products.html'">
20             Products
21         </button>
22     </div>
23 </div>
24
25 <h2 class="section-title">Orders</h2>
26
27 <div id="order-list" class="admin-grid"></div>
28
29 <button class="logout-btn" onclick="logout()">
30     Logout
31 </button>
32
33 </div>
34
35 <script src="admin-orders.js"></script>
36
37 </body>
38 </html>

```

admin-order.js

```

1     const API_URL = "http://localhost:3000/api";
2
3     const userRole = localStorage.getItem("userRole");
4
5     if (userRole !== "admin") {
6         alert("Access denied. Admin only.");
7         window.location.href = "index.html";
8     }
9
10    function createOrderCard(order) {
11
12        const paymentStatus = order.payment
13            ? order.payment.status
14            : "Completed";
15
16        const paymentAmount = order.payment
17            ? order.payment.amount
18            : order.totalAmount;
19
20        let itemsHtml = "";
21
22        if (order.items && order.items.length > 0) {
23
24            order.items.forEach(item => {
25
26                itemsHtml += `
27                    <li>
28                        ${item.name}
29                        x${item.quantity}
30                        - ${item.price}
31                    </li>
32                `;
33            });
34        }
35
36        return `
37            <div class="product-card">
38
39                <h3>Order #${order.id}</h3>
40
41                <p><strong>User ID:</strong> ${order.userId}</p>
42
43                <p><strong>Status:</strong> ${order.status}</p>

```

```

44
45     <p><strong>Payment:</strong> ${paymentStatus}</p>
46
47     <p><strong>Total:</strong> ${paymentAmount}</p>
48
49     <h4>Order Items</h4>
50
51     <ul class="order-items">
52         ${itemsHtml}
53     </ul>
54
55     ${order.reason ? `
56         <p style="color:red;">
57             <strong>Reject Reason:</strong>
58             ${order.reason}
59         </p>
60     ` : ""}
61
62     ${order.status === "Pending Approval" ? `
63
64         <button onclick="approveOrder(${order.id})">
65             Approve
66         </button>
67
68         <button onclick="rejectOrder(${order.id})">
69             Reject
70         </button>
71
72     ` : `
73
74     <p>
75         <strong>Final Status:</strong>
76         ${order.status}
77     </p>
78
79     `}
80 </div>
81 `;
82 }
83
84 function loadOrders() {
85
86     fetch(`${API_URL}/admin/orders`)
87     .then(res => res.json())
88     .then(orders => {
89
90         const orderList =
91             document.getElementById("order-list");
92
93         orderList.innerHTML = "";
94
95         if (!orders || orders.length === 0) {
96
97             orderList.innerHTML =
98                 "<p>No orders yet</p>";
99
100             return;
101         }
102
103         const pendingOrders = orders.filter(order =>
104             order.status === "Pending Approval"
105         );
106
107         const historyOrders = orders.filter(order =>
108             order.status !== "Pending Approval"
109         );
110

```

```

111     let pendingHtml = `
112         <h2 class="section-title">
113             Pending Orders
114         </h2>
115
116         <div class="admin-grid">
117     `;
118
119     if (pendingOrders.length === 0) {
120
121         pendingHtml += `
122             <p>No pending orders.</p>
123         `;
124     }
125
126     pendingOrders.forEach(order => {
127
128         pendingHtml += createOrderCard(order);
129     });
130
131     pendingHtml += `</div>`;
132
133     let historyHtml = `
134         <h2 class="section-title">
135             Order History
136         </h2>
137
138         <div class="admin-grid">
139     `;
140
141     if (historyOrders.length === 0) {
142
143         historyHtml += `
144             <p>No order history.</p>
145         `;
146     }
147
148     historyOrders.forEach(order => {
149
150         historyHtml += createOrderCard(order);
151     });
152
153     historyHtml += `</div>`;
154
155     orderList.innerHTML =
156         pendingHtml + historyHtml;
157     });
158 }
159
160 function approveOrder(orderId) {
161
162     fetch(`${API_URL}/admin/orders/${orderId}`, {
163         method: "PUT"
164     })
165
166     .then(res => res.json())
167
168     .then(data => {
169
170         alert(data.message);
171
172         loadOrders();
173     });
174 }
175
176 function rejectOrder(orderId) {
177

```

```

178     const reason = prompt("Enter reject reason:");
179
180     if (!reason) {
181
182         alert("Reject reason is required");
183
184         return;
185     }
186
187     fetch(`${API_URL}/admin/orders/${orderId}/reject`, {
188
189         method: "PUT",
190
191         headers: {
192             "Content-Type": "application/json"
193         },
194
195         body: JSON.stringify({
196             reason
197         })
198     })
199
200     .then(res => res.json())
201
202     .then(data => {
203
204         alert(data.message);
205
206         loadOrders();
207     });
208 }
209
210 function logout() {
211
212     localStorage.clear();
213
214     window.location.href = "index.html";
215 }
216
217 loadOrders();

```

admin-products.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale
6          =1.0">
7      <title>Admin Products</title>
8      <link rel="stylesheet" href="style.css">
9  </head>
10 <body>
11 <div class="container">
12
13     <div class="admin-header">
14
15         <div class="admin-title">
16             Admin Product Management
17         </div>
18
19         <div class="admin-nav">
20
21             <button
22                 id="orders-btn"
23                 onclick="window.location.href='admin-orders.html'">

```

```

24
25         Orders
26
27         </button>
28
29     </div>
30
31 </div>
32
33 <h2 class="section-title">
34     Add Product
35 </h2>
36
37 <form id="add-product-form">
38
39     <input
40         type="text"
41         id="product-name"
42         placeholder="Product Name"
43         required
44     >
45
46     <input
47         type="number"
48         id="product-price"
49         placeholder="Price"
50         required
51     >
52
53     <input
54         type="number"
55         id="product-stock"
56         placeholder="Stock"
57         required
58     >
59
60     <button type="submit">
61         Add Product
62     </button>
63
64 </form>
65
66 <h2 class="section-title">
67     Products
68 </h2>
69
70 <div
71     id="admin-product-list"
72     class="admin-grid">
73 </div>
74
75 <button
76     class="logout-btn"
77     onclick="logout()">
78
79     Logout
80
81 </button>
82
83 </div>
84
85 <script src="admin-products.js"></script>
86
87 </body>
88 </html>

```


admin-products.js

```
1  const API_URL = "http://localhost:3000/api";
2
3  const userRole = localStorage.getItem("userRole");
4
5  if (userRole !== "admin") {
6
7      alert("Access denied. Admin only.");
8
9      window.location.href = "index.html";
10 }
11
12 function loadAdminProducts() {
13
14     fetch(`${API_URL}/products`)
15
16         .then(res => res.json())
17
18         .then(products => {
19
20             const list =
21                 document.getElementById(
22                     "admin-product-list"
23                 );
24
25             list.innerHTML = "";
26
27             products.forEach(product => {
28
29                 list.innerHTML += `
30
31                     <div class="product-card">
32
33                         <h3>
34                             ${product.name}
35                         </h3>
36
37                         <p>
38                             Price: ${product.price}
39                         </p>
40
41                         <p>
42                             Stock: ${product.stock}
43                         </p>
44
45                         <input
46                             type="number"
47                             id="stock-${product.id}"
48                             placeholder="New Stock"
49                         >
50
51                         <button
52                             onclick="updateStock(${product.id})">
53
54                             Update Stock
55
56                         </button>
57
58                         <button
59                             onclick="deleteProduct(${product.id})">
60
61                             Delete Product
62
63                         </button>
64
65                     </div>
66                 `;
```

```

67         });
68     });
69 }
70
71 function loadPendingOrderCount() {
72     fetch(`${API_URL}/admin/orders`)
73         .then(res => res.json())
74         .then(orders => {
75             const pendingCount = orders.filter(order =>
76                 order.status ===
77                 "Pending Approval"
78             ).length;
79             const ordersBtn =
80                 document.getElementById(
81                     "orders-btn"
82                 );
83             if (pendingCount > 0) {
84                 ordersBtn.innerHTML =
85                     `Orders (${pendingCount})`;
86             } else {
87                 ordersBtn.innerHTML =
88                     `Orders`;
89             }
90         });
91 }
92
93 document.getElementById("add-product-form")
94     .addEventListener("submit", function(e) {
95         e.preventDefault();
96         const name =
97             document.getElementById(
98                 "product-name"
99             ).value;
100         const price = Number(
101             document.getElementById(
102                 "product-price"
103             ).value
104         );
105         const stock = Number(
106             document.getElementById(
107                 "product-stock"
108             ).value
109         );
110         fetch(`${API_URL}/products`, {
111             method: "POST",
112             headers: {

```

```

134         "Content-Type": "application/json"
135     },
136
137     body: JSON.stringify({
138         name,
139         price,
140         stock
141     })
142 })
143
144 .then(res => res.json())
145
146 .then(data => {
147
148     alert(data.message);
149
150     loadAdminProducts();
151
152     loadPendingOrderCount();
153
154     document.getElementById(
155         "add-product-form"
156     ).reset();
157 });
158 });
159
160 function deleteProduct(id) {
161
162     fetch(`${API_URL}/products/${id}`, {
163
164         method: "DELETE"
165     })
166
167 .then(res => res.json())
168
169 .then(data => {
170
171     alert(data.message);
172
173     loadAdminProducts();
174 });
175 }
176
177 function updateStock(productId) {
178
179     const stock = document.getElementById(
180         `stock-${productId}`
181     ).value;
182
183     fetch(`${API_URL}/products/${productId}/stock`, {
184
185         method: "PUT",
186
187         headers: {
188             "Content-Type": "application/json"
189         },
190
191         body: JSON.stringify({
192             stock
193         })
194     })
195
196 .then(res => res.json())
197
198 .then(data => {
199
200     alert(data.message);

```

```

201         loadAdminProducts();
202     });
203 }
204
205
206 function logout() {
207     localStorage.clear();
208     window.location.href = "index.html";
209 }
210
211 loadAdminProducts();
212
213 loadPendingOrderCount();
214
215

```

4.4 Backend Code

4.4.1 Database.js

```

1     const Database = require("better-sqlite3");
2
3     const db = new Database("database.db");
4
5     db.exec(`
6     CREATE TABLE IF NOT EXISTS users (
7         id INTEGER PRIMARY KEY AUTOINCREMENT,
8         name TEXT NOT NULL,
9         email TEXT UNIQUE NOT NULL,
10        password TEXT NOT NULL,
11        securityAnswer TEXT,
12        role TEXT DEFAULT 'customer'
13    );
14
15    CREATE TABLE IF NOT EXISTS products (
16        id INTEGER PRIMARY KEY AUTOINCREMENT,
17        name TEXT NOT NULL,
18        price REAL NOT NULL,
19        stock INTEGER NOT NULL
20    );
21
22    CREATE TABLE IF NOT EXISTS carts (
23        id INTEGER PRIMARY KEY AUTOINCREMENT,
24        userId INTEGER NOT NULL,
25        productId INTEGER NOT NULL,
26        quantity INTEGER NOT NULL
27    );
28
29    CREATE TABLE IF NOT EXISTS orders (
30        id INTEGER PRIMARY KEY AUTOINCREMENT,
31        userId INTEGER NOT NULL,
32        status TEXT NOT NULL,
33        reason TEXT,
34        totalAmount REAL NOT NULL
35    );
36
37    CREATE TABLE IF NOT EXISTS order_items (
38        id INTEGER PRIMARY KEY AUTOINCREMENT,
39        orderId INTEGER NOT NULL,
40        productId INTEGER NOT NULL,
41        name TEXT NOT NULL,
42        price REAL NOT NULL,
43        quantity INTEGER NOT NULL
44    );

```

```

45
46 CREATE TABLE IF NOT EXISTS payments (
47     id INTEGER PRIMARY KEY AUTOINCREMENT,
48     orderId INTEGER NOT NULL,
49     amount REAL NOT NULL,
50     method TEXT NOT NULL,
51     status TEXT NOT NULL
52 );
53 `);
54
55 const admin = db.prepare(
56     "SELECT * FROM users WHERE email = ?"
57 ).get("admin@test.com");
58
59 if (!admin) {
60     db.prepare(`
61         INSERT INTO users
62         (name, email, password, securityAnswer, role)
63         VALUES (?, ?, ?, ?, ?)
64     `).run(
65         "Admin",
66         "admin@test.com",
67         "admin123",
68         "admin",
69         "admin"
70     );
71 }
72
73 const productCount = db.prepare(
74     "SELECT COUNT(*) AS count FROM products"
75 ).get();
76
77 if (productCount.count === 0) {
78
79     db.prepare(`
80         INSERT INTO products (name, price, stock)
81         VALUES (?, ?, ?)
82     `).run("Laptop", 1500, 5);
83
84     db.prepare(`
85         INSERT INTO products (name, price, stock)
86         VALUES (?, ?, ?)
87     `).run("Phone", 900, 3);
88
89     db.prepare(`
90         INSERT INTO products (name, price, stock)
91         VALUES (?, ?, ?)
92     `).run("Headphones", 100, 10);
93 }
94
95 module.exports = db;

```

4.4.2 Controller

orderController.js

```

1 const OrderService = require("../services/orderService");
2
3 function createOrder(req, res) {
4     const { userId } = req.body;
5
6     const result = OrderService.createOrder(userId);
7
8     res.status(result.status).json({
9         message: result.message,
10        order: result.order

```

```

11     });
12 }
13
14 function getUserOrders(req, res) {
15     const { userId } = req.params;
16
17     const orders = OrderService.getUserOrders(userId);
18
19     res.status(200).json(orders);
20 }
21
22 function getAllOrders(req, res) {
23     const orders = OrderService.getAllOrders();
24
25     res.status(200).json(orders);
26 }
27
28 function approveOrder(req, res) {
29     const { id } = req.params;
30
31     const result = OrderService.approveOrder(id);
32
33     res.status(result.status).json({
34         message: result.message,
35         order: result.order
36     });
37 }
38
39 function rejectOrder(req, res) {
40     const { id } = req.params;
41     const { reason } = req.body;
42
43     const result = OrderService.rejectOrder(id, reason);
44
45     res.status(result.status).json({
46         message: result.message,
47         order: result.order
48     });
49 }
50
51 function cancelOrder(req, res) {
52     const { id } = req.params;
53
54     const result = OrderService.cancelOrder(id);
55
56     res.status(result.status).json({
57         message: result.message,
58         order: result.order
59     });
60 }
61
62 module.exports = {
63     createOrder,
64     getUserOrders,
65     getAllOrders,
66     approveOrder,
67     rejectOrder,
68     cancelOrder
69 };

```

4.4.3 Models

orderModel.js

```

1     const db = require("../database/database");
2

```

```

3 function createOrder(userId, cartItems, payment) {
4     const totalAmount = payment.amount;
5
6     const orderStmt = db.prepare(`
7         INSERT INTO orders (userId, status, reason, totalAmount)
8         VALUES (?, ?, ?, ?)
9     `);
10
11    const orderResult = orderStmt.run(
12        userId,
13        "Pending Approval",
14        null,
15        totalAmount
16    );
17
18    const orderId = orderResult.lastInsertRowid;
19
20    const itemStmt = db.prepare(`
21        INSERT INTO order_items
22        (orderId, productId, name, price, quantity)
23        VALUES (?, ?, ?, ?, ?)
24    `);
25
26    cartItems.forEach(item => {
27        itemStmt.run(
28            orderId,
29            item.id,
30            item.name,
31            item.price,
32            item.quantity
33        );
34    });
35
36    db.prepare(`
37        UPDATE payments
38        SET orderId = ?
39        WHERE id = ?
40    `).run(orderId, payment.id);
41
42    return getOrderById(orderId);
43 }
44
45 function getOrderById(orderId) {
46     const order = db.prepare(`
47         SELECT * FROM orders
48         WHERE id = ?
49     `).get(orderId);
50
51     if (!order) {
52         return null;
53     }
54
55     const items = db.prepare(`
56         SELECT
57             productId AS id,
58             name,
59             price,
60             quantity
61         FROM order_items
62         WHERE orderId = ?
63     `).all(orderId);
64
65     const payment = db.prepare(`
66         SELECT * FROM payments
67         WHERE orderId = ?
68     `).get(orderId);
69

```

```

70     return {
71         ...order,
72         items,
73         payment
74     };
75 }
76
77 function getOrdersByUser(userId) {
78     const orders = db.prepare(`
79         SELECT * FROM orders
80         WHERE userId = ?
81     `).all(userId);
82
83     return orders.map(order => getOrderById(order.id));
84 }
85
86 function getAllOrders() {
87     const orders = db.prepare(`
88         SELECT * FROM orders
89     `).all();
90
91     return orders.map(order => getOrderById(order.id));
92 }
93
94 function approveOrder(orderId) {
95     const order = getOrderById(orderId);
96
97     if (!order) {
98         return null;
99     }
100
101     if (order.status !== "Pending Approval") {
102         return false;
103     }
104
105     db.prepare(`
106         UPDATE orders
107         SET status = ?
108         WHERE id = ?
109     `).run("Approved", orderId);
110
111     return getOrderById(orderId);
112 }
113
114 function rejectOrder(orderId, reason) {
115     const order = getOrderById(orderId);
116
117     if (!order) {
118         return null;
119     }
120
121     if (order.status !== "Pending Approval") {
122         return false;
123     }
124
125     db.prepare(`
126         UPDATE orders
127         SET status = ?, reason = ?
128         WHERE id = ?
129     `).run("Rejected", reason, orderId);
130
131     return getOrderById(orderId);
132 }
133
134 function cancelOrder(orderId) {
135     const order = getOrderById(orderId);
136

```



```

137     if (!order) {
138         return null;
139     }
140
141     if (order.status !== "Pending Approval") {
142         return false;
143     }
144
145     db.prepare(`
146         UPDATE orders
147         SET status = ?
148         WHERE id = ?
149     `).run("Cancelled by Customer", orderId);
150
151     return getOrderById(orderId);
152 }
153
154 module.exports = {
155     createOrder,
156     getOrdersByUser,
157     getAllOrders,
158     approveOrder,
159     rejectOrder,
160     cancelOrder
161 };

```

paymentModel.js

```

1     const db = require("../database/database");
2
3     function processPayment(orderId, amount, method = "Fake Card") {
4         const stmt = db.prepare(`
5             INSERT INTO payments (orderId, amount, method, status)
6             VALUES (?, ?, ?, ?)
7         `);
8
9         const result = stmt.run(orderId, amount, method, "Completed");
10
11         return {
12             id: result.lastInsertRowid,
13             orderId,
14             amount,
15             method,
16             status: "Completed"
17         };
18     }
19
20     function getPaymentById(orderId) {
21         return db.prepare(`
22             SELECT * FROM payments
23             WHERE orderId = ?
24         `).get(orderId);
25     }
26
27     module.exports = {
28         processPayment,
29         getPaymentById
30     };

```

4.4.4 Services

authservice.js

```

1     const UserModel = require("../models/userModel");
2

```

```

3 function register(name, email, password, securityAnswer) {
4     const existingUser = UserModel.findUserByEmail(email);
5
6     if (existingUser) {
7         return {
8             status: 400,
9             message: "User already exists"
10        };
11    }
12
13    const user = UserModel.createUser(name, email, password,
14        securityAnswer);
15
16    return {
17        status: 201,
18        message: "Registration successful",
19        user
20    };
21 }
22
23 function login(email, password) {
24     const user = UserModel.findUserByEmail(email);
25
26     if (!user || user.password !== password) {
27         return {
28             status: 401,
29             message: "Invalid email or password"
30        };
31    }
32
33    return {
34        status: 200,
35        message: "Login successful",
36        user
37    };
38 }
39
40 function forgotPassword(email, securityAnswer, newPassword) {
41     const user = UserModel.findUserByEmail(email);
42
43     if (!user) {
44         return {
45             status: 404,
46             message: "User not found"
47        };
48    }
49
50    if (user.securityAnswer !== securityAnswer) {
51        return {
52            status: 401,
53            message: "Security answer is incorrect"
54        };
55    }
56
57    UserModel.updatePassword(email, newPassword);
58
59    return {
60        status: 200,
61        message: "Password updated successfully"
62    };
63 }
64
65 module.exports = {
66     register,
67     login,
68     forgotPassword
69 };

```

cartservice.js

```
1   const CartModel = require("../models/cartModel");
2   const ProductModel = require("../models/productModel");
3
4   function addToCart(userId, productId) {
5       const product = ProductModel.getProductById(productId);
6
7       if (!product) {
8           return {
9               status: 404,
10              message: "Product not found"
11          };
12      }
13
14      if (product.stock <= 0) {
15          return {
16              status: 400,
17              message: "Product is out of stock"
18          };
19      }
20
21      CartModel.addToCart(userId, product);
22
23      ProductModel.decreaseStock(productId, 1);
24
25      return {
26          status: 200,
27          message: "Product added to cart"
28      };
29  }
30
31  function getCart(userId) {
32      return CartModel.getCart(userId);
33  }
34
35  function removeFromCart(userId, productId) {
36      const removedItem = CartModel.removeFromCart(userId, productId);
37
38      if (!removedItem) {
39          return {
40              status: 404,
41              message: "Product not found in cart"
42          };
43      }
44
45      ProductModel.increaseStock(productId, 1);
46
47      return {
48          status: 200,
49          message: "Product removed from cart"
50      };
51  }
52
53  module.exports = {
54      addToCart,
55      getCart,
56      removeFromCart
57  };
```

orderservice.js

```
1   const OrderModel = require("../models/orderModel");
2   const PaymentModel = require("../models/paymentModel");
3   const CartModel = require("../models/cartModel");
```

```

4
5 function createOrder(userId) {
6     const cart = CartModel.getCart(userId);
7
8     if (!cart || cart.length === 0) {
9         return {
10             status: 400,
11             message: "Cart is empty"
12         };
13     }
14
15     let totalAmount = 0;
16
17     cart.forEach(item => {
18         totalAmount += item.price * item.quantity;
19     });
20
21     const temporaryOrderId = Date.now();
22
23     const payment = PaymentModel.processPayment(
24         temporaryOrderId,
25         totalAmount,
26         "Fake Card"
27     );
28
29     const order = OrderModel.createOrder(userId, cart, payment);
30
31     CartModel.clearCart(userId);
32
33     return {
34         status: 201,
35         message: "Order created successfully with fake payment",
36         order
37     };
38 }
39
40 function getUserOrders(userId) {
41     return OrderModel.getOrdersByUser(userId);
42 }
43
44 function getAllOrders() {
45     return OrderModel.getAllOrders();
46 }
47
48 function approveOrder(orderId) {
49     const order = OrderModel.approveOrder(orderId);
50
51     if (order === null) {
52         return {
53             status: 404,
54             message: "Order not found"
55         };
56     }
57
58     if (order === false) {
59         return {
60             status: 400,
61             message: "Only pending orders can be approved"
62         };
63     }
64
65     return {
66         status: 200,
67         message: "Order approved successfully",
68         order
69     };
70 }

```

```

71 function rejectOrder(orderId, reason) {
72     const order = OrderModel.rejectOrder(orderId, reason);
73
74     if (order === null) {
75         return {
76             status: 404,
77             message: "Order not found"
78         };
79     }
80
81     if (order === false) {
82         return {
83             status: 400,
84             message: "Only pending orders can be rejected"
85         };
86     }
87
88     return {
89         status: 200,
90         message: "Order rejected successfully",
91         order
92     };
93 }
94
95 function cancelOrder(orderId) {
96     const order = OrderModel.cancelOrder(orderId);
97
98     if (order === null) {
99         return {
100             status: 404,
101             message: "Order not found"
102         };
103     }
104
105     if (order === false) {
106         return {
107             status: 400,
108             message: "Only pending orders can be cancelled"
109         };
110     }
111
112     return {
113         status: 200,
114         message: "Order cancelled successfully",
115         order
116     };
117 }
118
119 module.exports = {
120     createOrder,
121     getUserOrders,
122     getAllOrders,
123     approveOrder,
124     rejectOrder,
125     cancelOrder
126 };
127

```

productservice.js

```

1     const ProductModel = require("../models/productModel");
2
3     function getProducts() {
4         return ProductModel.getProducts();
5     }
6
7     function addProduct(name, price, stock) {

```

```

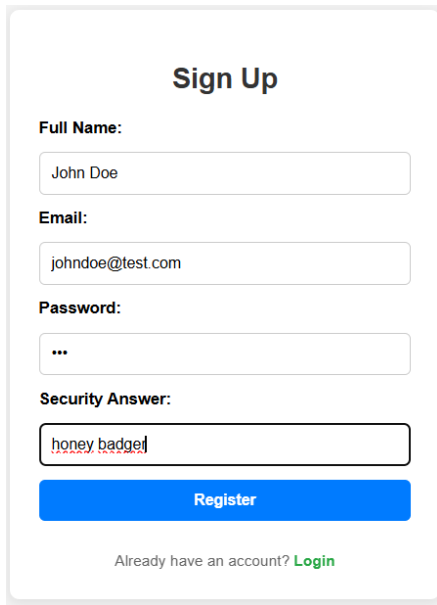
8     const product = ProductModel.addProduct(name, price, stock);
9
10    return {
11        status: 201,
12        message: "Product added successfully",
13        product
14    };
15 }
16
17 function deleteProduct(id) {
18     const deleted = ProductModel.deleteProduct(id);
19
20     if (!deleted) {
21         return {
22             status: 404,
23             message: "Product not found"
24         };
25     }
26
27     return {
28         status: 200,
29         message: "Product deleted successfully"
30     };
31 }
32
33 function updateStock(id, stock) {
34     const product = ProductModel.updateStock(id, stock);
35
36     if (!product) {
37         return {
38             status: 404,
39             message: "Product not found"
40         };
41     }
42
43     return {
44         status: 200,
45         message: "Stock updated successfully",
46         product
47     };
48 }
49
50 module.exports = {
51     getProducts,
52     addProduct,
53     deleteProduct,
54     updateStock
55 };

```

5 Web Application

5.1 User Login

Users can register and log in to the system, and are directed according to their roles. Authentication is controlled via the backend.



Sign Up

Full Name:

Email:

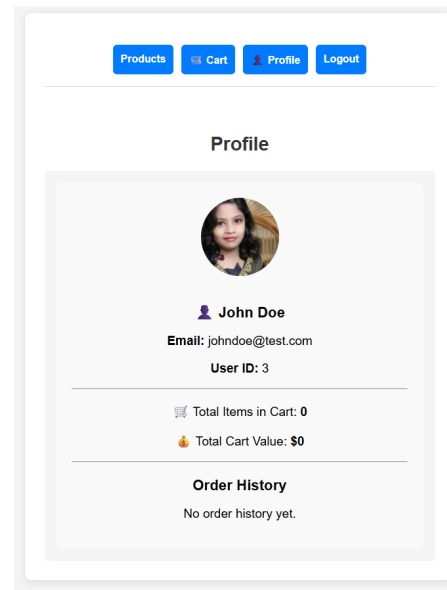
Password:

Security Answer:

[Register](#)

Already have an account? [Login](#)

(a) User Sign Up



Products Cart Profile Logout

Profile



John Doe

Email: johndoe@test.com

User ID: 3

Total Items in Cart: 0

Total Cart Value: \$0

Order History

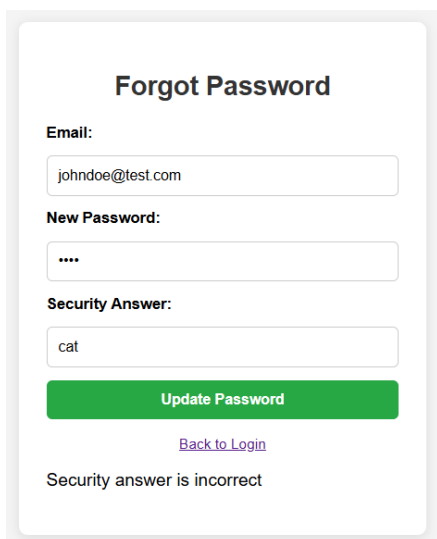
No order history yet.

(b) User Profile

Figure 8: User Interface Screens

5.2 forgot password & security answer

If users forget their password, they can set a new password by entering the answer to a security question. This prevents unauthorized access.



Forgot Password

Email:

New Password:

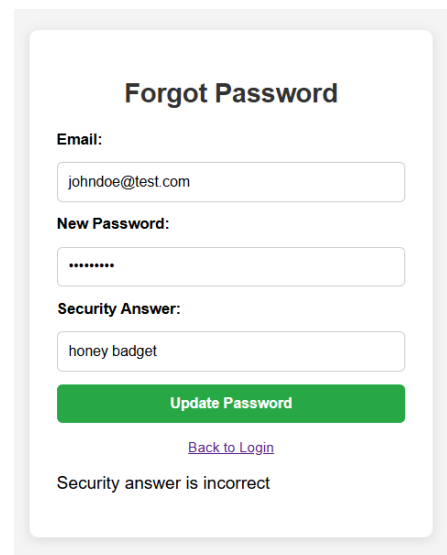
Security Answer:

[Update Password](#)

[Back to Login](#)

Security answer is incorrect

(a) Incorrect Security Answer



Forgot Password

Email:

New Password:

Security Answer:

[Update Password](#)

[Back to Login](#)

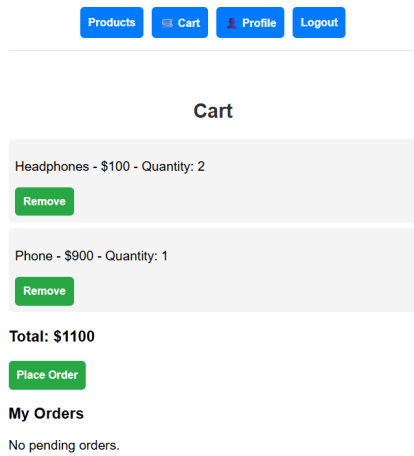
Security answer is incorrect

(b) Password Updated Successfully

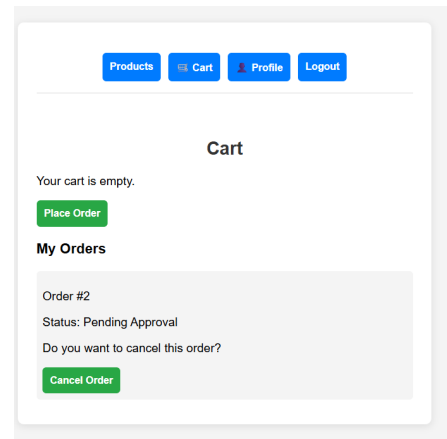
Figure 9: Forgot Password Process

5.3 Place order

The user creates an order for the products in their cart, and the system performs a simulated payment process.



(a) Place Order Step 1

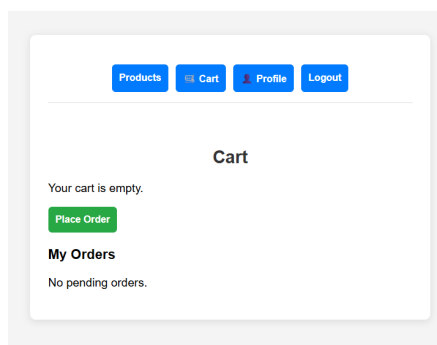


(b) Place Order Step 2

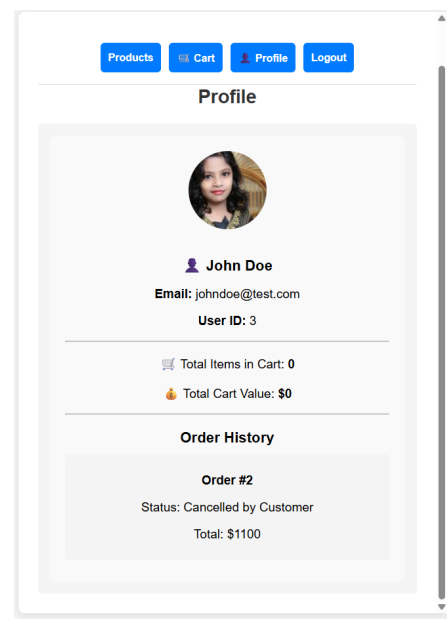
Figure 10: Order Placement Process

5.4 Pending order or cancel order

Orders that have not yet been confirmed are shown as "Pending," and the user can cancel this order if desired.



(a) Order Cancellation Step 1



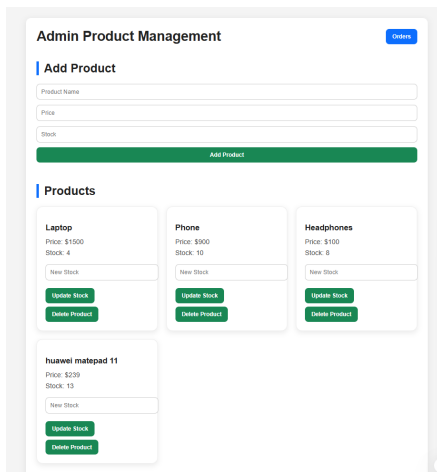
(b) Order Cancellation Step 2

Figure 11: Order Cancellation Process

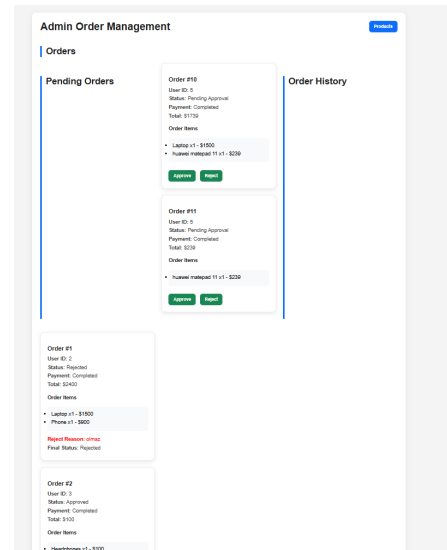
5.5 Admin Login

5.6 Admin product management

Admin users can add, delete products, and update stock information.



(a) Admin Product Management

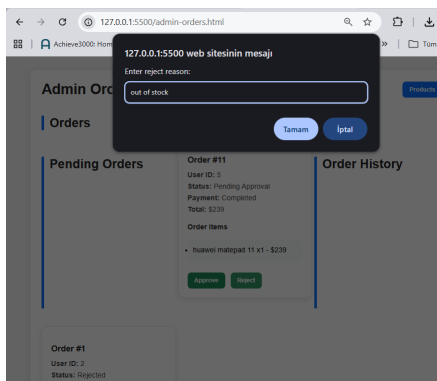


(b) Order History

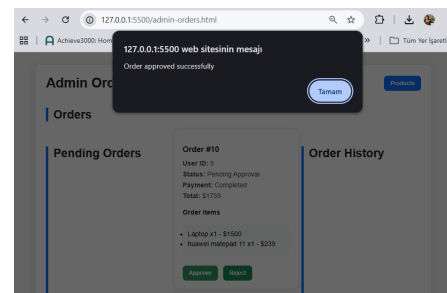
Figure 12: Admin Product Operations

5.7 Admin approve/reject order + reason

Admins can approve or reject orders. If rejected, a reason is sent to the user.



(a) Order Rejected

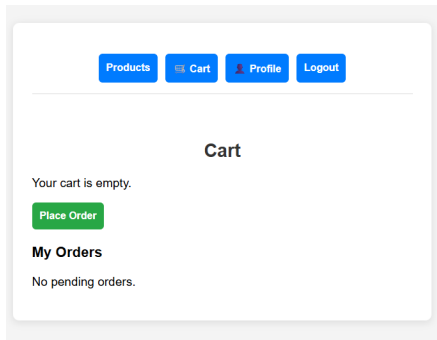


(b) Order Approved

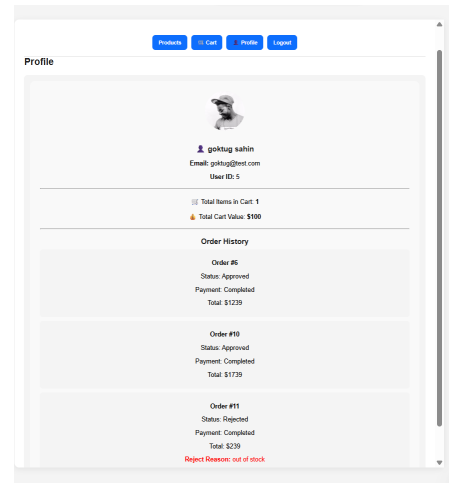
Figure 13: Admin Order Decision Process

5.8 Approved/rejected/canceled history in Customer profile

Users can see the approved, rejected, or canceled status of their orders on their profile page.



(a) Final Order Screen



(b) User Order History

Figure 14: Order Completion and History View